Bash Scripting -1

To view which shell we are using we use $0

```
faisal@Linux:/$ echo $0
/usr/bin/bash
```

To Check the shells in our server

```
faisal@Linux:/$ cat /etc/shells
# /etc/shells: valid login shells
/bin/sh
/usr/bin/sh
/bin/bash
/usr/bin/bash
/bin/rbash
/usr/bin/rbash
/usr/bin/dash
/usr/bin/screen
```

To see which user is assigned which shell we should see list of users in passwd file

```
faisal@Linux:/$ cat /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
```

For bash scripts create a directory scripts and use this folder for all the bash file

```
[root@ip-172-31-35-122 /]# mkdir script
[root@ip-172-31-35-122 /]# ls
bin  boot  dev  etc  home  lib  lib64  local  media  mnt  opt  proc  root  run  sbin  script  scripts  srv  sys  tmp  usr  var
```

Now for the first bash scripts we will use basic linux commands, create a file with .bash extension

```
[root@ip-172-31-35-122 /]# ls
bin  boot  dev  etc  home  lib  lib64  local  media  mnt  opt  proc  root  run  sbin  scripts  srv  sys  tmp  usr  var
[root@ip-172-31-35-122 /]# cd scripts
[root@ip-172-31-35-122 scripts]# vi basic.bash
```

1. **Basic Script:** Add #!/bin/bash so that kernel can interpret it and complete the task

```
root@ip-172-31-35-
#!/bin/bash
pwd
ls
whoami
date
cal
touch a b c
```

To execute the scripts we should either change the file permission or use bash command to use it

```
[root@ip-172-31-35-122 scripts]# chmod 755 basic.bash
[root@ip-172-31-35-122 scripts]# ./basic.bash
/scripts
a  access.bash  admin.bash  b  basic.bash  c  case.bash  filepath.bash
root
Fri Oct 16 11:29:33 UTC 2026
    October 2026
Su Mo Tu We Th Fr Sa
             1  2  3
 4  5  6  7  8  9 10
11 12 13 14 15 16 17
18 19 20 21 22 23 24
25 26 27 28 29 30 31
```

```
[root@ip-172-31-35-122 scripts]# bash ./basic.bash
/scripts
basic.bash
root
Fri Oct 16 10:05:09 UTC 2026
    October 2026
Su Mo Tu We Th Fr Sa
             1  2  3
 4  5  6  7  8  9 10
11 12 13 14 15 16 17
18 19 20 21 22 23 24
25 26 27 28 29 30 31
```

2. Admin script:- We add commands which help us see cpu usage, and stats of system

```
[root@ip-172-31-35-122 scripts]# vi admin.bash
[root@ip-172-31-35-122 scripts]# cat admin.bash
#!/bin/bash

top | head -10
df -h
free -m
uptime
iostat
[root@ip-172-31-35-122 scripts]#
```

Then running the bash file with bash command

```
[root@ip-172-31-35-122 scripts]# bash ./admin.bash
top - 10:06:59 up 26 min,  2 users,  load average: 0.00, 0.00, 0.00
Tasks: 107 total,   1 running, 106 sleeping,   0 stopped,   0 zombie
%Cpu(s):  3.2 us,  0.0 sy,  0.0 ni, 96.8 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem :    913.1 total,    489.4 free,    167.0 used,    256.7 buff/cache
MiB Swap:      0.0 total,      0.0 free,      0.0 used.    614.3 avail Mem

    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
   4373 root      20   0  224104   3592   2852 R   6.7   0.4   0:00.01 top
      1 root      20   0  173968  17868  11236 S   0.0   1.9   0:01.17 systemd
      2 root      20   0       0      0      0 S   0.0   0.0   0:00.00 kthreadd
Filesystem      Size  Used Avail Use% Mounted on
devtmpfs        4.0M     0  4.0M   0% /dev
tmpfs           457M     0  457M   0% /dev/shm
tmpfs           183M  444K  183M   1% /run
efivarfs        128K  2.7K  121K   3% /sys/firmware/efi/efivars
/dev/nvme0n1p1  8.0G  1.6G  6.4G  20% /
tmpfs           457M     0  457M   0% /tmp
/dev/nvme0n1p128   10M  1.3M  8.7M  13% /boot/efi
tmpfs            92M     0   92M   0% /run/user/1000
               total        used        free      shared  buff/cache   available
Mem:             913         166         489           0         256         614
Swap:              0           0           0
 10:06:59 up 26 min,  2 users,  load average: 0.00, 0.00, 0.00
Linux 6.18.54-121.166.amzn2023.x86_64 (ip-172-31-35-122.ap-southeast-2.compute.internal)        10/16/26        _x86_64_        (2 CPU)

avg-cpu:  %user   %nice %system %iowait  %steal   %idle
          0.58    0.00    0.26    0.07    0.14   98.95

Device            tps    kB_read/s    kB_wrtn/s    kB_dscd/s    kB_read    kB_wrtn    kB_dscd
nvme0n1          4.53       101.22       131.48         0.00     159179     206764          0
```

3. Variables:- Instead of directly printing something we use variable to store command or data. This variables can be accessed with $varname

```
[root@ip-172-31-35-122 scripts]# vi variable.bash
[root@ip-172-31-35-122 scripts]# cat variable.bash
#!/bin/bash
#Define Variables

p=pwd
l=ls
wi=whoami
d=date
c='cal 2026'

echo run variable tasks
$p
$l
$wi
$date
$c
[root@ip-172-31-35-122 scripts]#
```

Running the created bash file with bash command

```
[root@ip-172-31-35-122 scripts]# bash ./variable.bash
run variable tasks
/scripts
a  admin.bash  b  basic.bash  c  variable.bash
root
                              2026

       January               February                March
Su Mo Tu We Th Fr Sa   Su Mo Tu We Th Fr Sa   Su Mo Tu We Th Fr Sa
             1  2  3    1  2  3  4  5  6  7      1  2  3  4  5  6  7
 4  5  6  7  8  9 10    8  9 10 11 12 13 14      8  9 10 11 12 13 14
11 12 13 14 15 16 17   15 16 17 18 19 20 21     15 16 17 18 19 20 21
18 19 20 21 22 23 24   22 23 24 25 26 27 28     22 23 24 25 26 27 28
25 26 27 28 29 30 31                            29 30 31

        April                  May                   June
Su Mo Tu We Th Fr Sa   Su Mo Tu We Th Fr Sa   Su Mo Tu We Th Fr Sa
          1  2  3  4                   1  2         1  2  3  4  5  6
 5  6  7  8  9 10 11    3  4  5  6  7  8  9      7  8  9 10 11 12 13
12 13 14 15 16 17 18   10 11 12 13 14 15 16     14 15 16 17 18 19 20
19 20 21 22 23 24 25   17 18 19 20 21 22 23     21 22 23 24 25 26 27
26 27 28 29 30         24 25 26 27 28 29 30     28 29 30
                       31

        July                 August                September
Su Mo Tu We Th Fr Sa   Su Mo Tu We Th Fr Sa   Su Mo Tu We Th Fr Sa
          1  2  3  4                      1            1  2  3  4  5
 5  6  7  8  9 10 11    2  3  4  5  6  7  8      6  7  8  9 10 11 12
12 13 14 15 16 17 18    9 10 11 12 13 14 15     13 14 15 16 17 18 19
19 20 21 22 23 24 25   16 17 18 19 20 21 22     20 21 22 23 24 25 26
26 27 28 29 30 31      23 24 25 26 27 28 29     27 28 29 30
                       30 31

       October              November              December
Su Mo Tu We Th Fr Sa   Su Mo Tu We Th Fr Sa   Su Mo Tu We Th Fr Sa
             1  2  3    1  2  3  4  5  6  7            1  2  3  4  5
 4  5  6  7  8  9 10    8  9 10 11 12 13 14      6  7  8  9 10 11 12
11 12 13 14 15 16 17   15 16 17 18 19 20 21     13 14 15 16 17 18 19
18 19 20 21 22 23 24   22 23 24 25 26 27 28     20 21 22 23 24 25 26
25 26 27 28 29 30 31   29 30                    27 28 29 30 31
```

4. Input and Output: - We have a syntax read to take input from user and to print the text we have echo

root@ip-172-31-35-122:/scripts

```
[root@ip-172-31-35-122 scripts]# vi input.bash
[root@ip-172-31-35-122 scripts]# cat input.bash
#!/bin/bash

echo Hello, my name is Faisal
echo
echo what is your name?
read name
echo
echo Hello $name
echo
```

Now run this file with bash the output will stop at read line and ask us input after it takes input and stores it in variable we can print it

```
[root@ip-172-31-35-122 scripts]# bash ./input.bash
Hello, my name is Faisal

what is your name?
Techie

Hello Techie

[root@ip-172-31-35-122 scripts]#
```

- Sometimes when we pass linux commands in variables it print as text and does not work so we use Backticks (` `) to this commands

```
[root@ip-172-31-35-122 scripts]# vi int-out.bash
[root@ip-172-31-35-122 scripts]# cat int-out.bash
#!/bin/bash

#we need to use the Backticks if the command is taken as a text or else not required

a=`hostname`
echo Hello, my name is $a
echo
echo what is your name?
read b
echo
echo Hello $b
echo
```

After using backticks in the script we can use the command

```
[root@ip-172-31-35-122 scripts]# bash ./int-out.bash
Hello, my name is ip-172-31-35-122.ap-southeast-2.compute.internal

what is your name?
faisal

Hello faisal

[root@ip-172-31-35-122 scripts]#
```

5. If-else:- We use if-else to print a text on a specific condition.

```
[root@ip-172-31-35-122 scripts]# vi if-else.bash
[root@ip-172-31-35-122 scripts]# cat if-else.bash
#!/bin/bash
#using the if else to check the count

count=100
if [ $count -eq 100 ]
then
        echo count is 100
else
        echo count is not 100
fi
```

After running the command we get the output according to condition if count is equal the it will print then part of the script. If not matching else part will print

To end the if-else loop we use fi command

```
[root@ip-172-31-35-122 scripts]# bash ./if-else.bash
count is 100
```